# 클라우드 엔지니어링 6기

멋쟁이사자처럼

26.04.02 (목) | 정범석님

## 자바 OOP, 클래스, 생성자 정리

### 👉 오늘의 강의 학습 요약

금일 강의는 2차원 배열(정방형/비정방형) 생성 방식 3가지와 비정방형에서의 동적 열 생성, 그리고 `main(String[] args)`의 커맨드라인 아규먼트 활용, `Random` API 사용을 복습한 뒤, 자바의 객체지향(OOP) 개념으로 본격 전환되었습니다. 현실 세계의 객체를 자바에서 클래스-인스턴스-멤버(변수/메소드)로 매핑하는 관점을 중심으로, "재사용 클래스(메인 없음)"와 이를 실행/생성하는 "핸들링 클래스(메인 있음)"의 2클래스 구조를 반복적으로 강조하였습니다.

이후 접근지정자(특히 `public`/`private`)로 외부 접근을 통제하는 이유(보안/무결성)와, `private`로 막힌 인스턴스 변수에 값을 넣기 위한 우회로로서 생성자(기본 생성자, 파라미터 생성자, 오버로딩)를 상세히 다뤘습니다. 생성자 호출 규칙(개수/타입/순서 일치), `this` 키워드(필드 식별, 생성자 체이닝), IDE를 통한 생성자 자동 생성까지 연결하여 실습(고양이/학생 관리 예제)로 정리하였고, 마지막으로 메소드의 개념/문법/생성자와의 차이 및 호출 방식까지 개요를 제시했습니다.

## 1. 2차원 배열 복습과 생성 패턴

> **2차원 배열은 정방형(모든 행의 열 크기 동일)과 비정방형(행마다 열 크기 상이)으로 나뉘며, `new`, 리터럴, `new+리터럴` 의 3가지 방식으로 생성할 수 있고 비정방형은 열을 동적으로 생성해야 합니다.**

2차원 배열의 기본 타입 표기는 대괄호가 2개이며, 첫 번째 차원은 행 크기, 두 번째 차원은 열 크기를 의미합니다. 정방형(직사각형) 배열은 모든 행의 열 크기가 같으므로, `new` 로 생성할 때 열 크기를 함께 지정하면 전체 행에 동일한 열 크기가 일괄로 만들어집니다.

정방형에서 리터럴 초기화는 중괄호 중첩으로 표현되며, 바깥 중괄호의 원소 개수(안쪽 중괄호 개수)가 행 크기가 됩니다. 각 안쪽 중괄호 내부 값의 개수가 열 크기이며, 모든 행의 값 개수가 같으면 정방형으로 해석됩니다.

`new` 와 리터럴을 혼합하는 방식은 "리터럴로 값을 주는 경우"이므로 크기를 명시하면 안 된다는 규칙을 강조하였습니다. 즉, 리터럴을 쓰는 순간 배열 크기는 값으로부터 유추되며, 별도 크기 지정은 금지됩니다.

비정방형(가변 길이 2차원 배열)은 행 크기만 먼저 만들고, 각 행의 열(1차원 배열)을 별도의 `new` 로 동적으로 생성해야 합니다. 따라서 `new` 가 2번 등장하며, 접근은 `arr[행][열]` 형태로 동일하지만 각 행마다 열 길이가 다를 수 있습니다.

| 구분 | 핵심 생성 방식 | 크기 지정 규칙 | 특징 |
|---|---|---|---|
| 정방형 | `new int[행][열]` 또는 리터럴 | `new` 는 행/열 지정 가능, 리터럴은 값으로 크기 결정 | 모든 행의 열 크기가 동일합니다. |
| 비정방형 | `new int[행][]` 후 각 행에 `new int[열]` | 처음부터 열 크기를 지정하면 비정방형 취지에 맞지 않습니다. | 행마다 열 크기를 다르게 생성합니다. |
| `new+리터럴` | `new int[][] { ... }` | 리터럴 사용 시 크기 명시는 금지됩니다. | 값 기반 초기화와 `new` 표기를 혼합합니다. |

**핵심 키워드** #2차원배열 #정방형배열 #비정방형배열 #리터럴초기화 #동적열생성

## 2. 커맨드라인 아규먼트와 Random API 복습

> **`main(String[] args)` 의 `args` 는 실행 시 전달값을 문자열 배열로 받으며, 숫자 처리에는 `Integer.parseInt` 가 필요하고, `Random` 은 객체 생성 후 `nextInt` 로 범위 내 난수를 얻습니다.**

자바 프로그램 실행 시 `main(String[] args)` 의 `args` 파라미터로 외부 입력을 전달할 수 있습니다. 전달된 값은 공백 기준으로 분리되어 `String[]` 에 저장되며, 첫 값은 `args[0]` , 다음 값은 `args[1]` 처럼 순서대로 매핑됩니다.

전달값은 숫자로 입력하더라도 "문자열 배열"로 받기 때문에 기본적으로 모두 문자열로 저장됩니다. 숫자 계산이 필요하면 문자열을 정수로 변환해야 하며, 이때 `Integer.parseInt` 를 사용합니다.

공백이 포함된 문장을 하나의 인자로 전달하려면 실행 인자에서 쌍따옴표로 묶어야 하며, IDE(이클립스)에서도 실행 설정에서 아규먼트를 지정할 수 있음을 언급하였습니다.

`Random` 클래스는 자바 표준 API로 제공되는 클래스이며, 사용하려면 반드시 객체 생성( `new` )으로 메모리에 올려야 합니다. 객체가 생성되면 참조변수와 점( `.` ) 연산자로 메소드에 접근하여 난수를 얻습니다.

```
Random r = new Random();
int x = r.nextInt(bound); // 0 이상 bound 미만
```

난수 범위의 기본 형태로 `nextInt(bound)` 를 자주 사용하며, 0부터 `bound-1` 까지의 값을 반환하는 점을 강조하였습니다. 필요 시 "어디서부터 어디까지" 형태의 범위 난수도 만들 수 있으나, 통상은 위 형태가 가장 빈번히 쓰입니다.

**핵심 키워드** `#main메소드` `#커맨드라인아규먼트` `#parseInt` `#Random` `#nextInt`

## 3. OOP와 객체, 클래스, 인스턴스 매핑

> **현실 세계의 객체는 속성과 동작으로 구성되며, 자바에서는 객체를 클래스로, 속성을 인스턴스 변수로, 동작을 메소드로 매핑하여 OOP를 구현합니다.**

객체지향 프로그래밍(OOP)은 "객체(Object)를 중심으로 프로그램을 구성하는 방법론"으로 정리됩니다. 현실 세계에서 주체(관찰자)가 바라보는 사물(노트북, 고양이, 학생 등)을 객체로 보고, 각 객체는 반드시 속성(특성)과 동작(행위)을 가진다는 점이 핵심입니다.

자바에서 현실 객체를 표현할 때는 "가상 세계의 최소 단위가 클래스"이므로 객체는 클래스로 표현됩니다. 또한 객체의 속성은 변수로, 객체의 동작은 메소드로 표현되며, 이때 속성을 담는 변수는 주로 인스턴스 변수로 다룹니다.

현업 개발 관점에서는 애플리케이션 요구사항에서 필요한 객체를 "추출"하고, 그 객체의 수많은 속성/동작 중 필요한 것만 "솎아내는" 과정이 중요합니다. 이 과정을 추상화(Abstraction) 또는 모델링(Modeling)으로 설명하였으며, 프로젝트/협업에서 가장 빈번히 수행되는 설계 활동임을 강조하였습니다.

| 현실 세계 개념 | 자바 표현 | 예시(고양이) |
|---|---|---|
| 객체(Object) | 클래스(Class) | `Cat` |
| 속성(Attribute) | 인스턴스 변수(멤버 변수) | `name` , `age` , `gender` |
| 동작(Behavior) | 메소드(멤버 메소드) | `eat()` , `cry()` |

또한 객체/클래스/인스턴스는 맥락에 따라 혼용되기 쉬우나, "객체는 현실 개념", "클래스는 설계도(틀)", "인스턴스는 클래스 기반으로 힙에 생성된 실체"로 구별해두면 혼란을 줄일 수 있습니다. 붕어빵 틀(클래스)과 붕어빵(인스턴스) 비유로 다수 인스턴스 생성 개념을 정리하였습니다.

**핵심 키워드** #OOP #객체 #클래스 #인스턴스 #추상화

## 4. 재사용 클래스와 핸들링 클래스, 객체 생성과 멤버 접근

> **메인 메소드가 없는 재사용 클래스는 단독 실행이 불가하므로, 메인 메소드가 있는 핸들링 클래스에서 `new` 로 객체를 생성하고 참조변수 `.` 로 멤버에 접근합니다.**

객체에 대응되는 클래스(예: `Cat` )는 재사용을 전제로 하므로 일반적으로 `main` 메소드를 두지 않습니다. `main` 이 있으면 실행 로직이 고정되어 재사용성이 떨어지고, 애플리케이션은 원칙적으로 시작점이 되는 `main` 이 하나여야 한다는 점을 함께 설명하였습니다.

따라서 "메인이 없는 클래스(재사용 클래스)"와 이를 실행/테스트하는 "메인이 있는 클래스(핸들링 클래스)"의 2 클래스 구조가 기본이 됩니다. 핸들링 클래스는 재사용 클래스를 객체 생성하여 메모리(힙)에 올리고, 참조변수로 멤버에 접근해 사용합니다.

객체 생성 문법은 아래 형태로 정리되며, `Cat` 처럼 클래스 타입 변수는 참조형 변수로서 주소값을 저장합니다.

```
Cat c1 = new Cat();
```

메모리 관점에서는 참조변수 `c1` 은 스택에 생성되고, `new Cat` 로 만들어진 인스턴스(필드/메소드가 포함된 구조)는 힙에 생성됩니다. 인스턴스 변수는 자동 초기화되며(예: `String` 은 `null` , `int` 는 `0` ), 참조가 끊기면(예: `null` 대입 또는 다른 인스턴스를 가리킴) GC 대상이 될 수 있음을 연결하여 복습했습니다.

멤버 접근은 "참조변수.멤버" 형태로 수행하며, 여기서 멤버는 인스턴스 변수 또는 메소드입니다. 예제에서는 `Cat` 인스턴스의 `name/age/gender` 를 설정하고 `eat()/cry()` 를 호출함으로써 "실제 개체(야옹이/뭉치)를 프로그램으로 관리한다"는 흐름을 확인했습니다.

| 구성 요소 | 용어 | 비고 |
|---|---|---|
| 인스턴스 변수 | 멤버 변수 | 객체의 속성을 저장합니다. |
| 메소드 | 멤버 메소드 | 객체의 동작을 구현합니다. |
| 생성자 | 구성 요소이지만 멤버로는 보지 않음 | 인스턴스 생성 시 초기화에 관여합니다. |

**핵심 키워드** #객체생성 #new #참조변수 #멤버접근 #자동초기화

## 5. 접근지정자(public/private)와 캡슐화 필요성

> **접근지정자는 클래스 간 접근 가능 여부를 제어하며, 특히 인스턴스 변수는 외부의 잘못된 직접 접근을 막기 위해 일반적으로 `private` 로 선언합니다.**

클래스가 여러 개로 나뉘면 한 클래스가 다른 클래스의 변수/메소드에 접근하는 상황이 발생하며, 이를 제어하는 문법이 접근지정자(Access Modifier)입니다. 접근을 "완전히 열거나(공개)" "완전히 닫는(비공개)" 관점에서 우선 `public` 과 `private` 을 중심으로 다뤘습니다.

접근지정자는 클래스/변수(로컬 제외)/메소드/생성자에 지정할 수 있으며, 로컬 변수에는 붙일 수 없다는 제한을 짚었습니다. 접근 수준은 `public → protected → default(생략) → private` 순으로 제한이 강화된다는 흐름을 강조하였습니다.

직접 접근을 허용하면(예: `age` 를 외부에서 바로 수정) 문법적으로는 문제가 없더라도 논리적으로 말이 안 되는 값(예: 200살)이 저장될 수 있습니다. 이를 방지하기 위해 인스턴스 변수는 일반적으로 `private` 로 선언하여 외부 직접 접근을 차단하고, 값 설정/검증은 다른 경로(생성자/메소드)로 제공해야 한다는 결론으로 이어졌습니다.

| 지정자 | 핵심 의미 | 강의에서의 포커스 |
|---|---|---|
| `public` | 제한이 거의 없는 공개입니다. | 클래스/메소드에서 자주 보던 기본 형태입니다. |
| `private` | 외부 접근을 강하게 차단합니다. | 인스턴스 변수 보호와 검증 경로 마련이 목적입니다. |

**핵심 키워드** #접근지정자 #public #private #캡슐화 #데이터무결성

## 6. 생성자: 기본 생성자, 파라미터, 오버로딩, 검증

> **생성자는 인스턴스 생성 시점에 인스턴스 변수를 초기화하며, 파라미터(매개변수)와 아규먼트(인자)는 개수, 타입, 순서가 일치해야 하고, 생성자를 명시하면 기본 생성자는 자동 제공되지 않습니다.**

생성자는 `public 클래스명(...) { ... }` 형태로 선언되며, 인스턴스가 생성될 때(즉 `new` 시점) 호출되어 인스턴스 변수 초기화를 담당합니다. `private` 로 막힌 인스턴스 변수에 대해서도 생성자 내부에서는 초기화가 가능하므로, "직접 접근은 막되 초기화는 허용"하는 캡슐화 전략의 첫 번째 해결책으로 제시되었습니다.

생성자 호출은 `new 클래스명(인자들)` 로 이뤄지며, 파라미터(받는 쪽 변수)와 아규먼트(호출 시 전달값)의 매핑은 **개수, 데이터 타입, 순서**가 모두 일치해야 합니다. 불일치하면 컴파일 에러가 발생하며, 이를 "아규먼트 리스트 매칭 규칙"으로 정리하였습니다.

개발자가 생성자를 명시적으로 작성하지 않으면 자바가 "기본 생성자(디폴트 생성자)"를 자동으로 제공하여 `new 클래스명()` 호출을 받을 수 있게 합니다. 그러나 생성자를 하나라도 명시하면 기본 생성자는 더 이상 자동 제공되지 않으므로, 기본 형태로 생성하려면 기본 생성자를 직접 작성해야 합니다.

생성자는 여러 개 만들 수 있으며, 같은 이름(클래스명)이라도 파라미터 구성이 다르면 구별되므로 오버로딩이 성립합니다. 오버로딩은 "필요한 정보만 알고 있는 상황"을 지원하기 위해 사용하며, 이름/나이/성별을 모두 아는 경우, 일부만 아는 경우, 아무것도 모르는 경우를 각각 다른 생성자로 초기화할 수 있습니다.

생성자를 쓰는 장점으로는 (1) 초기화 과정에서 검증 로직을 넣을 수 있고, (2) 인스턴스 생성 시점에 빠르게 초기화가 가능하다는 점을 비교 사례로 설명하였습니다. 즉, 직접 접근은 검증 기회 없이 값이 저장되지만, 생성자 기반 초기화는 "저장 전 검증"이 가능해집니다.

| 항목 | 직접 접근 초기화 | 생성자 초기화 |
|---|---|---|
| 검증 가능성 | 값이 바로 들어가 검증 기회가 부족합니다. | 초기화 전에 조건 검증 로직을 넣을 수 있습니다. |
| 초기화 시점 | 생성 후 별도 대입으로 초기화됩니다. | `new` 시점에 초기화가 진행됩니다. |

**핵심 키워드** #생성자 #기본생성자 #오버로딩 #파라미터 #아규먼트

## 7. this 키워드와 생성자 체이닝(this())

> **`this` 는 힙에 생성된 "현재 인스턴스 자신"을 참조하며, 필드와 로컬 변수 이름 충돌 시 `this.` 로 필드를 지정하고, `this()` 로 생성자 간 호출을 하여 중복 초기화를 제거합니다.**

`this` 는 인스턴스 자신의 주소(자기 자신 참조)를 의미하며, 생성자/메소드 내부에서 "현재 인스턴스의 멤버"를 명시할 때 사용됩니다. 파라미터 이름을 필드명과 동일하게 주면 가독성은 좋아지지만, 단순히 `name = name;` 처럼 쓰면 가까운 범위의 로컬 변수를 참조하여 자기 자신에게 대입하는 형태가 되어 필드 초기화가 되지 않습니다.

이때 `this.name = name;` 처럼 `this.` 를 붙이면 좌변이 인스턴스 변수(필드)임이 명확해지고, 우변은 로컬 변수(파라미터)로 해석되어 원하는 초기화가 이루어집니다.

또한 `this()` 는 같은 클래스 내 다른 생성자를 호출하는 문법으로, 오버로딩 생성자들 사이의 초기화 로직 중복을 제거하는 목적을 가집니다. 예를 들어 (name, age) 생성자가 (name, age, gender) 생성자를 호출하여 "초기화는 한 곳에서만" 처리되도록 구성할 수 있습니다.

`this()` 호출은 생성자 내부에서만 가능하며, 반드시 생성자의 첫 줄(first line)에 와야 한다는 제약을 함께 정리하였습니다. 생성자 외부(일반 메소드 등)에서 사용하거나 첫 줄이 아니면 컴파일 에러가 발생합니다.

| 사용 형태 | 의미 | 핵심 제약 |
|---|---|---|
| `this.필드` | 현재 인스턴스의 멤버(필드/메소드) 접근입니다. | 이름 충돌 시 필드 식별에 중요합니다. |
| `this(...)` | 같은 클래스의 다른 생성자 호출입니다. | 생성자 내부에서만, 첫 줄에서만 허용됩니다. |

IDE(이클립스)에서는 생성자를 직접 타이핑하지 않아도
`Source > Generate Constructor using Fields` 등으로 자동 생성할 수 있으며, 이때 파라미터명과 필드명이 같도록 만들고 `this.` 를 붙이는 형태가 일반적으로 생성됨을 확인했습니다.

**핵심 키워드** `#this` `#필드식별` `#this호출` `#생성자체이닝` `#중복제거`

## 8. 학생 관리 예제로 통합 실습 정리

> **학생 객체를 클래스(설계도)로 만들고, 여러 인스턴스를 생성해 서로 다른 데이터를 관리하며, 생성자 오버로딩과 `this()` 로 초기화 경로를 통합하고 출력까지 수행합니다.**

학생 관리 애플리케이션 요구를 가정하여 학생 객체를 추출하고, 속성으로 이름/나이/주소를 선정하여 `Student` 클래스를 정의했습니다. `Student` 는 재사용 클래스이므로 `main` 없이 만들고, `TestStudent` 같은 핸들링 클래스에서 객체 생성과 출력 로직을 수행하는 구조로 구성했습니다.

학생 3명을 관리하기 위해 동일한 클래스(틀)로부터 인스턴스 3개를 생성하며, 이때 각 인스턴스의 인스턴스 변수 값이 달라져 "동일 구조/상이 데이터"를 관리할 수 있음을 확인했습니다. 주소를 모르는 학생 같은 케이스를 위해 생성자 오버로딩(예: `(name, age)` 와 `(name, age, address)` )을 제공하고, `(name, age)` 생성자는 `this(name, age, null)` 로 체이닝하여 초기화를 한 곳에 몰아주는 형태를 실습했습니다.

출력은 인스턴스별로 필드를 참조해 포맷 출력으로 정리했으며, 프라이빗을 아직 적용하지 않은 이유는 "메소드(조회/수정)를 아직 배우기 전이라 출력 접근을 우선 허용"하기 위함임을 명확히 했습니다.

**핵심 키워드** `#학생관리` `#인스턴스여러개` `#생성자오버로딩` `#this체이닝` `#핸들링클래스`

## 9. 메소드 개요: 역할, 문법, 생성자와의 차이, 호출 방식

> **메소드는 기능 처리 단위로서 조회, 수정, 검증, 연산 등을 수행하며, 리턴 타입과 이름을 가지며, 객체 생성 후 참조변수 `.` 로 호출합니다.**

메소드는 객체의 동작(기능)을 담당하며, 검증/연산/수정/조회 등 다양한 처리를 수행할 수 있습니다. 강의에서는 특히 인스턴스 변수의 값을 **수정하거나 조회**하는 역할로 메소드를 활용할 예정임을 명시하였습니다.

메소드 기본 문법은 `public 반환타입 메소드명(파라미터 ...) { ... }` 로 정리되며, 파라미터는 있을 수도 없을 수도 있고, 리턴 또한 있을 수도 없을 수도 있습니다. 강의 노트에서 대괄호는 배열 의미가 아니라 "옵션(있어도/없어도 됨)"을 표시하는 표기라는 점을 분리하여 설명했습니다.

생성자와 비교했을 때 메소드는 (1) 리턴 타입이 반드시 존재하고, (2) 메소드명은 임의로 정할 수 있으며, (3) 리턴을 가질 수 있다는 점이 두드러진 차이로 제시되었습니다. 반면 생성자는 클래스명과 동일해야 하며 리턴 타입이 없습니다.

호출 방식도 구분됩니다. 생성자는 객체 생성 시 `new` 와 함께 호출되지만, 메소드는 객체를 생성한 뒤 참조변수 `.` 를 통해 호출합니다. 즉, 메소드 사용을 위해서도 인스턴스가 메모리에 올라가 있어야 하므로 객체 생성이 선행되어야 한다는 점을 강조했습니다.

| 구분 | 생성자 | 메소드 |
|---|---|---|
| 호출 시점 | `new` 시점에 호출됩니다. | 객체 생성 후 `참조변수.메소드()` 로 호출됩니다. |
| 이름 규칙 | 클래스명과 동일해야 합니다. | 의미 있는 임의의 이름을 사용할 수 있습니다. |
| 반환 타입 | 존재하지 않습니다. | 반드시 반환 타입이 표기됩니다. |

**핵심 키워드** #메소드 #리턴타입 #파라미터 #메소드호출 #기능처리